

TEXT ANALYTICS

Introduction to Language Models

Francesco Spinnato

* francesco.spinnato@unipi.it
University of Pisa



Language and Probability

Main idea: to assign a **probability** to a **linguistic structure**, on the basis of our previous experience.

- **Tokenization**

$$P(\text{"the car is parked by the station"}) > P(\text{"thec ar is parked by the station"})$$

- **PoS tagging**

$$P(\text{"he tried to } \underbrace{\text{bank}}_{\text{verb}} \text{ the plane"}) > P(\text{"he tried to } \underbrace{\text{bank}}_{\text{noun}} \text{ the plane"})$$

- **Speech recognition**

$$P(\text{"I saw a van"}) > P(\text{"eyes awe of an"})$$

- **Machine translation**

ITA: forti venti stanotte $\rightarrow$ ENG:

$$P(\text{"high winds tonight"}) > P(\text{"large winds tonight"})$$

Introduction to Language Models

Reminds on Probability

Where Do These Probabilities Come From?

We will never be given these probabilities. We have to **estimate them from text we have already seen**.

Frequentist interpretation. The probability approximates the *relative frequency* with which an event occurs, over a sufficiently large number of trials carried out under the same conditions.

If n is the number of repetitions of an experiment and f_A the number of times an event A is observed:

$$P(A) = \lim_{n \rightarrow \infty} \frac{f_A}{n}$$

By the **Law of Large Numbers**, the average over many trials approaches the expected value as the number of trials grows.

A Corpus Is an Experiment

A corpus C can be seen as the result of an experiment: randomly selecting a word, $|C|$ times.

- number of trials: $|C|$, the size of the corpus in tokens
- each corpus token is one outcome of that experiment
- for a word type w (e.g. "cat"), the frequency f_w counts how often it appears

$$P(w) \approx \frac{f_w}{|C|}$$

That is the whole estimation machinery for this part of the course: **probability is a count divided by a total**. Everything that follows is about *which* counts to take.

Two things that sound alike but are not:

- $P(E_1)$: the probability that a word in a **vocabulary** is a verb
- $P(E_2)$: the probability of **using** a verb in a language

$$P(E_1) \stackrel{?}{=} P(E_2)$$

Both are countable, but over different things:

$$P(E_1) = \frac{\#(\text{verbs in vocabulary})}{\#(\text{words in vocabulary})} \qquad P(E_2) = \frac{\#(\text{occurrences of verbs})}{\#(\text{words in text})}$$

$$P(E_1) \neq P(E_2)$$

A dictionary is a list of *types*; a text is a stream of *tokens*.

Language models model the *use* of language, estimated from large collections of text: so it is $P(E_2)$ that we are after.

And the questions we actually want to ask are of this shape:

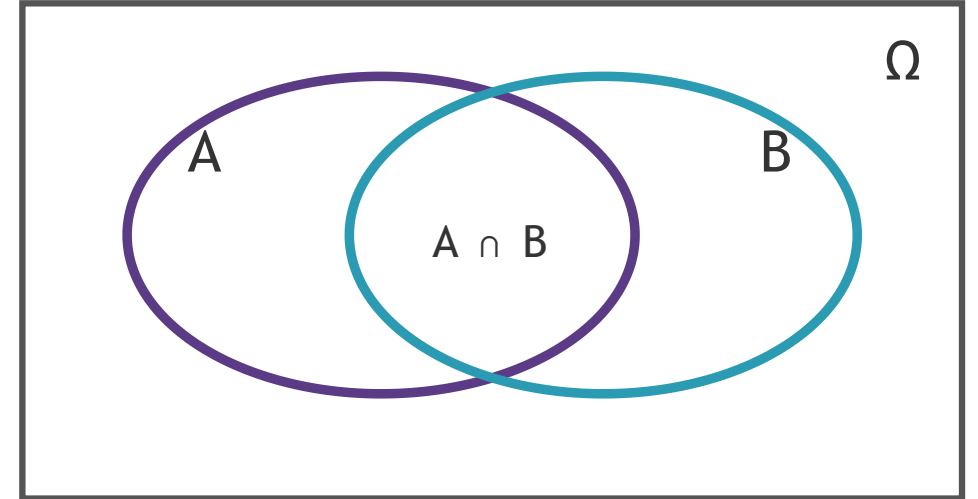
- What is the most probable word to appear after "*would*"?
- What is the probability of the letter "*h*" following the letter "*t*"?

$\implies$ **Conditional probabilities!**

Conditional Probabilities

A **conditional probability** $P(A \mid B)$ is the probability that A occurs once B has occurred.

- B is the set of outcomes granted to happen; any outcome outside B has probability 0.
- A and B are **dependent**: B conditions A .
- The question is how much A "covers" B : the higher the coverage, the higher the probability.



$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad \Longleftrightarrow \quad P(A \cap B) = P(A \mid B) \cdot P(B)$$

Why Conditioning Matters

Conditional probabilities let us improve a prediction whenever **partial information (evidence)** is available. They are the key tool for statistical inference:

- What is the probability of snow?
- What is the probability of snow in **Pisa**?
- What is the probability of snow in Pisa, **when Temp = 30 °C**?
- What is the most probable word to appear **after "would"**?
- What is the probability of the letter **"h" following the letter "t"**?
- What is the probability of a document containing *"president"*, *"elections"* and *"polls"* belonging to the topic *"cooking"*?

The last three are, respectively, a **language model**, a **character-level model**, and a **text classifier**.

Joint Probability

$P(A, B) = P(A \cap B)$ is the probability of event A and event B happening together:

- the probability of getting two 6s by tossing two dice
- the probability of a card being a four **and** red
- the probability of extracting a verb **and** a noun from a text

How it is computed depends on the relation between the two events:

- **Independent events:** the outcome of one does not influence the probability of the other.
- **Dependent events:** the outcome of one influences the other, so we resort to conditional probability.

Joint Probability (Independent Events)

The probability of two **independent** events is the product of their probabilities:

$$P(A, B) = P(A) \cdot P(B)$$

Ex. 1: throwing two dice and obtaining two 6s:

- $P(d_1 = 6) = 1/6$ and $P(d_2 = 6) = 1/6$
- $P(d_1 = 6, d_2 = 6) = 1/6 \cdot 1/6 = 1/36$

Ex. 2: selecting a verb and a noun from a randomly generated text:

- assuming $|C| = 80$, $f(\text{nouns}) = 15$, $f(\text{verbs}) = 20$
- $P(\text{noun}) = 15/80$ and $P(\text{verb}) = 20/80$
- $P(\text{verb}, \text{noun}) = 15/80 \cdot 20/80 = 3/64$

From Conditional to Joint Probability ...

The joint probability $P(A, B)$ is the probability of A and B occurring at the same time, i.e. $P(A, B) = P(A \cap B)$. So it can be written two ways:

$$P(A \mid B) = \frac{P(A, B)}{P(B)} \implies P(A, B) = P(A \mid B) \cdot P(B)$$

$$P(B \mid A) = \frac{P(A, B)}{P(A)} \implies P(A, B) = P(B \mid A) \cdot P(A)$$

$$\text{and therefore } P(B \mid A) \cdot P(A) = P(A \mid B) \cdot P(B)$$

Remember this identity. Applied to a sequence of words it becomes the **chain rule**, which is where the next part begins. Applied to a hypothesis and some evidence, it becomes Bayes' theorem.

... to the Bayes Theorem

From $P(B \mid A) \cdot P(A) = P(A \mid B) \cdot P(B)$ we can derive:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

Term	Name	Meaning
$P(A \mid B)$	Posterior probability	having seen B , we know more about the chance of A
$P(B \mid A)$	Likelihood	the degree of belief in B , having observed A
$P(A)$	Prior	the chance of A in the absence of any data
$P(B)$	Evidence	the probability of observing B

Bayes lets us flip a conditional. We often want $P(A \mid B)$ but can only estimate $P(B \mid A)$ from data. Bayes converts one into the other.

Bayes Theorem: An Example

Is an email **spam** (A), given that it contains the word *free* (B)?

Term	Value	Where it comes from
Prior $P(\text{spam})$	0.2	20% of all emails are spam
Likelihood $P(\text{free} \mid \text{spam})$	0.5	half of the spam emails contain <i>free</i>
Evidence $P(\text{free})$	0.14	14% of all emails contain <i>free</i>

$$P(\text{spam} \mid \text{free}) = \frac{P(\text{free} \mid \text{spam}) \cdot P(\text{spam})}{P(\text{free})} = \frac{0.5 \cdot 0.2}{0.14} \approx 0.71$$

- Observing "free" moves the spam probability from **20%** (prior) to **71%** (posterior).
- $P(\text{free} \mid \text{spam}) = 0.5$ does **not** mean $P(\text{spam} \mid \text{free}) = 0.5$: Bayes is what lets us flip the conditional correctly.
- The likelihood is easy to estimate: count how often *free* appears in emails labelled as spam.

Independence

A and B are **independent** if and only if the occurrence of one does not change the probability of the other:

$$P(A \mid B) = P(A) \quad \text{and} \quad P(B \mid A) = P(B)$$

Assuming independence drastically simplifies the modeling of complex objects such as text:

- tokens do not depend on previous tokens, e.g., $P(\text{like} \mid \text{would}) = P(\text{like})$, $P(\text{h} \mid \text{t}) = P(\text{h}) \rightarrow$ unigram language models;
- the **naïve Bayes** classifier uses exactly this word-independence assumption;
- modeling *some* degree of dependence gives more accurate models $\rightarrow$ the **n -gram language models**.

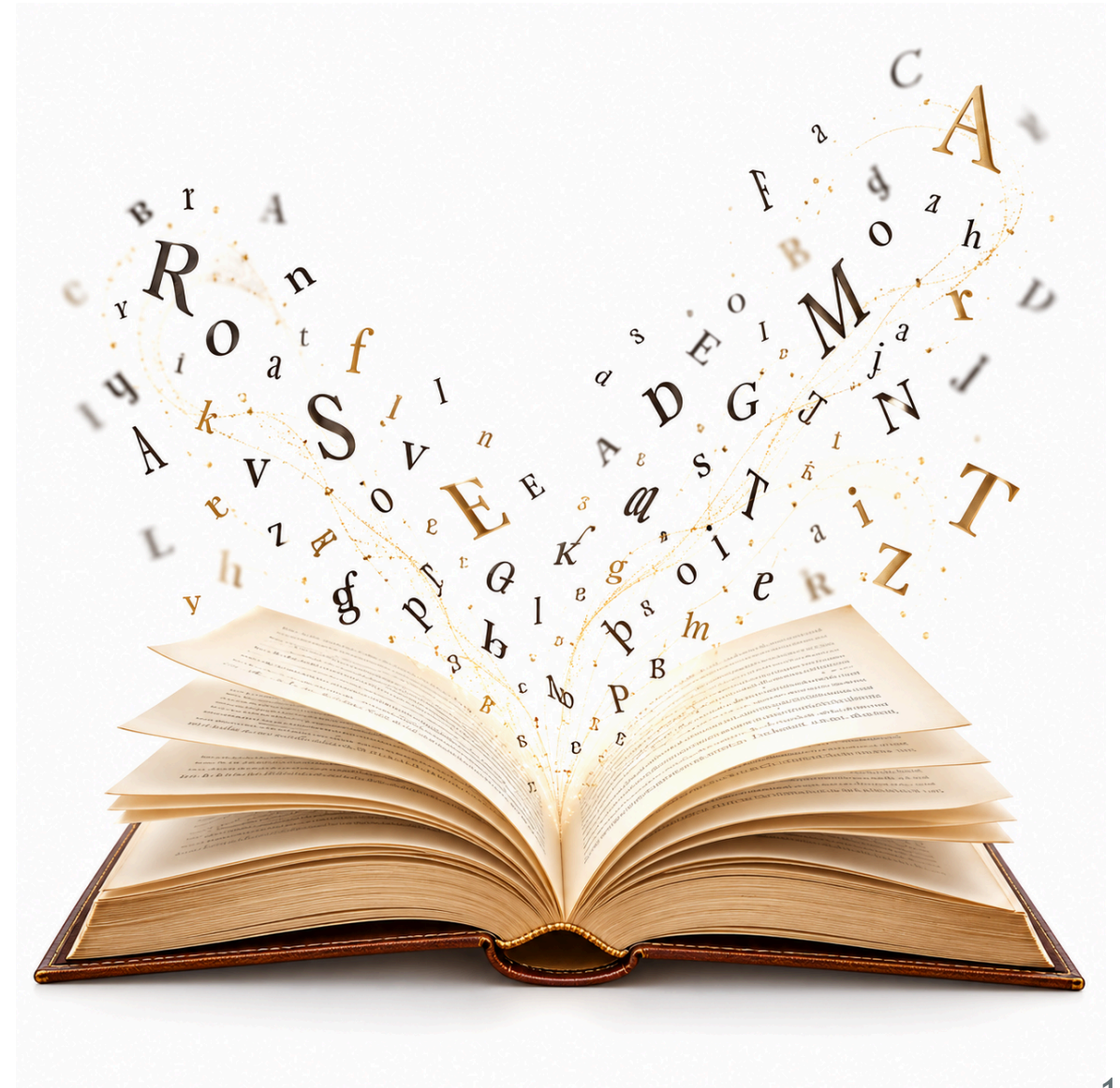
Introduction to Language Models

Probabilistic Language Models

Probabilistic Language Model

A probabilistic/statistical language model is a probability distribution P over sequences of words.

It assigns a probability to a whole document, not just to isolated words.



We write $P(\text{the})$ for the probability that a word in the text is “the”.

- A sequence of T words is a document (D) and is written $w_1 \dots w_T$, or $w_{1:T}$,
 - w_i is the word in position i .
- The probability of observing exactly this sequence is the joint probability $P(w_1, \dots, w_T)$.

Symbols used throughout this lecture:

C	the corpus (training collection); $ C $ its size in tokens
D	a document or text being scored; T its length in tokens
V	the vocabulary (a set); $ V $ its size
w_0	$= \langle s \rangle$, a sentence-start marker

The Chain Rule

Given a document D composed of a sequence of words w_1, w_2, w_3 , we can use the **chain rule** to decompose the probability of the document into the probabilities of its words:

$$P(D) = P(w_1 w_2 w_3 \dots) = P(w_1) P(w_2 \mid w_1) P(w_3 \mid w_1 w_2) \dots$$

The chain rule of probability is *always true*: it is a fundamental law of probability and makes **no assumptions** about word dependencies.

E.g.:

$$P(\text{Star Wars}) = P(\text{'A long time ago in a galaxy far, far away [...] The end.'}) = \\ P(\text{'A'}) P(\text{'long'} \mid \text{'A'}) P(\text{'time'} \mid \text{'A long'}) P(\text{'ago'} \mid \text{'A long time'}) \dots$$

The Chain Rule

$$P(D) = P(w_1 w_2 w_3 \dots) = P(w_1) P(w_2 \mid w_1) P(w_3 \mid w_1 w_2) \dots$$

The formula above makes no assumptions and can **exactly** model any language.

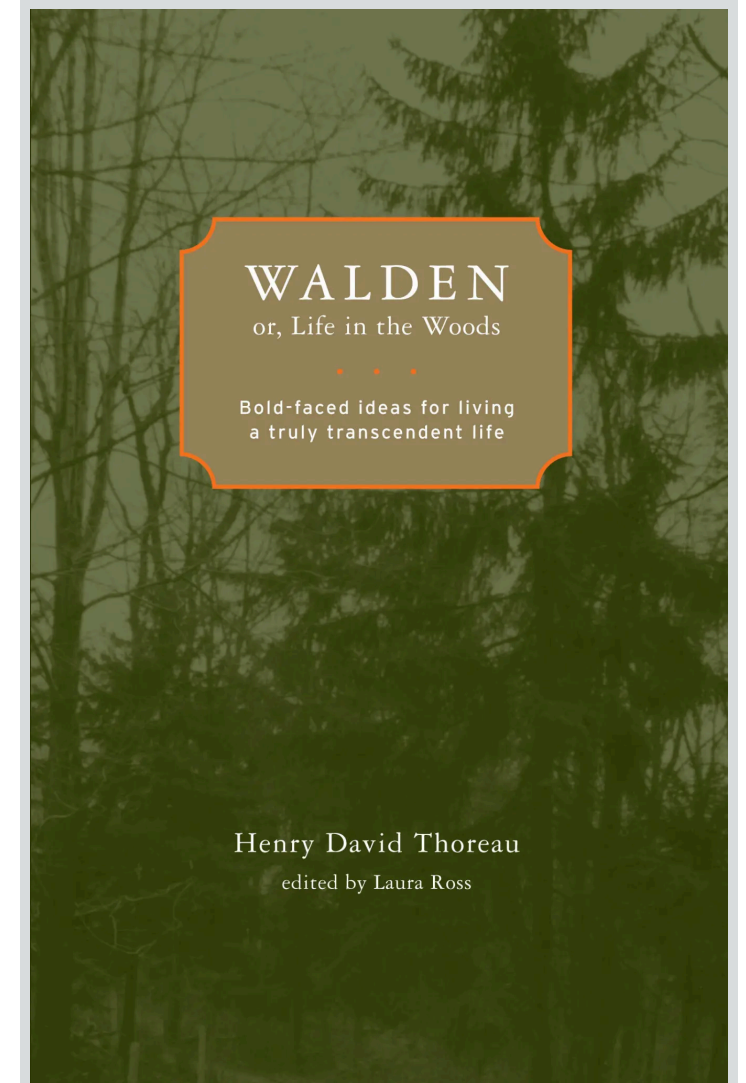
- Yet it is impractical, because it requires learning the probability of *any* sequence in the language.
- Even a single one of the factors in the formula is unreliable to compute

The probability of words

Let's compute the probability of a word w given some history of previous words, $P(w_i \mid w_1 \dots w_{i-1})$.

Suppose the history is “*The water of Walden Pond is so beautifully*” and we want to know the probability that the next word is *blue*:

$P(\text{blue} \mid \text{The water of Walden Pond is so beautifully})$



The probability of words

$$P(\text{blue} | \text{The water of Walden Pond is so beautifully})$$

One way to estimate this probability is directly from relative frequency counts, i.e., the ratio between how many times:

- "*The water of Walden Pond is so beautifully*" appears in the book and
- "*The water of Walden Pond is so beautifully blue*" appears in the book.

Unfortunately:

- the entire book isn't big enough to estimate counts of entire sentences;
- language is creative: new sentences are invented all the time;
- we can't expect to get reliable counts for such large objects as entire sentences.
- depending on the **assumptions** we make about the probability distribution, we can build statistical models of different complexity.

Unigrams Model

A **unigram model** assumes statistical independence between words, i.e., the probability of D is the product of the probabilities of its words:

$$\begin{aligned} P(D) &= P(w_1 w_2 w_3 \dots) = P(w_1) P(w_2 \mid w_1) P(w_3 \mid w_1 w_2) \dots \\ &\stackrel{\text{unigram}}{=} P(w_1) P(w_2) P(w_3) \dots = \prod_i P(w_i) \end{aligned}$$

E.g.:

$$P(\text{Star Wars}) \stackrel{\text{unigram}}{=} P('A') P('long') P('time') P('ago') \dots P('far') P('far') \dots P('The') P('end')$$

The unigram model assumption is that words are independent of each other: the unigram assumption loses all information about **word order**.

Counting

Probabilities are determined by **counting** over a collection of documents, the corpus C , from which we derive the parameters of our language model:

$$P(w) = \frac{\#(\text{occurrences of } w \text{ in } C)}{\#(\text{words in } C)} = \frac{\text{count}(w)}{\sum_{w' \in V} \text{count}(w')}$$

where $\text{count}(\cdot)$ counts occurrences in the corpus and V is the vocabulary, the set of unique words.

For example, given the corpus:

I am Sam Sam I am I do not like green eggs and ham

$$P('I') = \frac{\#('I' \text{ in the corpus})}{\#(\text{words in the corpus})} = \frac{3}{14} = 21.4\%$$

Maximum Likelihood Estimation

The ratio between the number of occurrences of a word and the total number of words in the corpus is a **maximum likelihood estimate**: the estimate that maximizes the probability of the training data under the model.

Maximum Likelihood Estimation: a method for estimating the parameters of a statistical model by maximizing the likelihood of some observed data.

If a word has a relative frequency of P in a given corpus, we assume that its probability in the language should be P as well.

Training and Scoring Unigram Models

Using a language model takes two steps:

1. **Train** on a corpus C : estimate every $P(w)$ by counting in C .
2. **Score** a new text D , which need not be in C .

E.g., trained on `I am Sam` `Sam I am` `I do not like green eggs and ham`, $|C| = 14$:

$$P(\text{'Sam am I'}) = P(\text{'Sam'}) P(\text{'am'}) P(\text{'I'}) = \frac{2}{14} \cdot \frac{2}{14} \cdot \frac{3}{14} \approx 0.0044$$

Critical issues? Try $D = \text{I like pizza}$.

$$P(\text{'I like pizza'}) = P(\text{'I'}) P(\text{'like'}) P(\text{'pizza'}) = \frac{3}{14} \cdot \frac{1}{14} \cdot \frac{0}{14} = 0$$

Critical Issues of Unigram Models

1. **Unseen words zero out the text.** One word missing from C gives $P(w) = 0$, and then $P(D) = 0$, as for `I like pizza` .
2. **Tiny numbers.** $P(D)$ is a product of T values below 1: a 1000-word text with $P(w_i) \approx 0.01 = 10^{-2}$ gives 10^{-2000} , which a computer rounds to 0.
3. **Word order is lost.** Every permutation of the same words gets the same probability:

$$P(\text{'Sam am I'}) = P(\text{'I am Sam'}) = P(\text{'am I Sam'}) \approx 0.0044$$

What if a word does not appear in the corpus C ?

$$P(w) = \frac{0}{|C|} = 0 \implies P(D) = 0$$

To solve this issue, words that are never observed in the collection used to infer the language model statistics can be given a **non-zero probability**...

- so that a single unknown word does not crash $P(D)$ to zero,
- so that texts with unknown words are given a lower probability than those with all-known words.

Smoothing methods add some probability mass to unknown events, taking it from known events.

Smoothing: Add-One (Laplace)

Add-one (Laplace, or additive) smoothing adds one occurrence to every event, including the unknown one:

$$P_{\text{Laplace}}(w) = \frac{\text{count}(w) + 1}{|C| + |V|} = \frac{\text{count}(w) + 1}{\sum_{w' \in V} \text{count}(w') + |V|}$$

where:

- $\text{count}(w)$ is the number of occurrences of word w in the corpus C
- $|C| = \sum_{w' \in V} \text{count}(w')$ is the total number of word tokens in the corpus
- $|V|$ is the vocabulary size (number of unique words)

Adding 1 may alter the probabilities of known events too much...

Smoothing: Add-k

Adding 1 may alter the probabilities for known events too much:

- especially when using small collections
- and it can give relatively too much importance to unknown events

We can choose to add a smaller value $k < 1$, e.g. $k = 0.01$:

$$P_{\text{Add-}k}(w) = \frac{\text{count}(w) + k}{\sum_{w' \in V} \text{count}(w') + k|V|} = \frac{\text{count}(w) + k}{|C| + k|V|}$$

Smoothing adds robustness with respect to unknown and rare events, but it is always preferable, when possible, to **add more text** to the collection to get stronger statistics.

Log Space Computation

Probabilities are often very small. To avoid numerical underflow and simplify computations, models are trained and combined in **log space**, especially when applying smoothing:

$$\log(P_1 P_2 P_3 P_4) = \log(P_1) + \log(P_2) + \log(P_3) + \log(P_4)$$

Advantages:

- avoids numbers smaller than machine precision (underflow errors)
- it becomes a linear model
- we can easily go back to probabilities at the end of the computation (exponentiation)
- marginal efficiency gain: summing is faster than multiplying (if the individual logs are precomputed)

From Unigrams to N-Grams

With unigrams we made an assumption of **total independence** between words, which is too harsh and leads to poor models.

Yet we know that considering **all** the dependencies between words is too complex and statistically fragile.

Can we find a balance between these two extremes?

Markov Property

In a stochastic process, the next event may depend on an **arbitrarily long sequence of past events**.

- In a stochastic process that satisfies the **(first-order) Markov property**, the next state depends **only on the current state** and is conditionally independent of all previous states.
- A **higher-order Markov property** states that the next state depends only on a fixed number of previous states (e.g. the last two or three).
- We can define a simple language model, more powerful than a unigram model, by making a **Markov assumption** about language.



The Markov Assumption

Instead of calculating the probability of a word given its entire history, we approximate it using just the **last few words**.

The Markov assumption: the probability of a word depends only on the last $n - 1$ words, where n is the n -gram order:

$$P(w_i \mid w_1 \dots w_{i-1}) \approx P(w_i \mid w_{i-n+1} \dots w_{i-1})$$

For instance, with history *"its water is so"*:

- $n = 1$ (**unigram**) $\Rightarrow P(\text{"transparent"})$
- $n = 2$ (**bigram**) $\Rightarrow P(\text{"transparent"} \mid \text{"so"})$
- $n = 3$ (**trigram**) $\Rightarrow P(\text{"transparent"} \mid \text{"is so"})$

Markov Chain Models

A k -th order Markov model assumes that each event depends only on a window of the k preceding events, which corresponds to a $(k + 1)$ -gram.

Higher-order models have more "memory", but are harder to estimate.

- **Zero-order Markov model:** events are independent, i.e. we use *unigrams*:

$$P(A_1, \dots, A_n) = P(A_1) \cdot P(A_2) \cdots P(A_n)$$

- **First-order Markov model:** each event depends on the preceding event, i.e. *bigrams*:

$$P(A_1, \dots, A_n) = P(A_1) \cdot P(A_2 \mid A_1) \cdots P(A_n \mid A_{n-1})$$

- **Second-order Markov model:** each event depends on the 2 preceding events, i.e. *3-grams*:

$$P(A_1, \dots, A_n) = P(A_1) \cdot P(A_2 \mid A_1) \cdot P(A_3 \mid A_1, A_2) \cdots P(A_n \mid A_{n-2}, A_{n-1})$$

Careful, two different counts: **Markov order** k = how many previous words we look at; **n-gram order** $n = k+1$ = how many words the gram contains. A bigram is a first-order Markov model.

Bigrams Model

A **bigram model** assumes statistical dependence of a word and the preceding one, i.e., $P(w_i \mid w_1 \dots w_{i-1}) \approx P(w_i \mid w_{i-1})$, thus:

$$\begin{aligned} P(D) &= P(w_1 w_2 w_3 \dots w_T) \approx P(w_1) P(w_2 \mid w_1) P(w_3 \mid w_2) \dots P(w_T \mid w_{T-1}) \\ &\stackrel{\text{bigram}}{=} \prod_{i=1}^T P(w_i \mid w_{i-1}) \end{aligned}$$

so that the product can start at $i = 1$, we set $w_0 = \langle s \rangle$, a **sentence-start marker**.

Example: $P(\text{I like pizza}) \approx P(\text{I} \mid \langle s \rangle) P(\text{like} \mid \text{I}) P(\text{pizza} \mid \text{like})$

- In this way, some information about **word order** is captured.
- This simple addition already captures a good amount of syntactic regularity.

Similarly to unigrams, using MLE, the conditional probability of a word w_i given the previous word w_{i-1} is:

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}w_i)}{\text{count}(w_{i-1})}$$

Bigram Model

For example, given `<s>` a sentence-start marker, and the corpus:

`<s> I am Sam` `<s> Sam I am` `<s> I do not like green eggs and ham`

The probability of the sequence `I am` is:

$$P(\text{I am}) = P(\text{I} \mid \langle s \rangle) P(\text{am} \mid \text{I}) = \frac{\text{count}(\langle s \rangle \text{ I})}{\text{count}(\langle s \rangle)} \frac{\text{count}(\text{I am})}{\text{count}(\text{I})} = \frac{2}{3} \frac{2}{3} = \frac{4}{9}$$

Similarly for `I am Sam`:

$$P(\text{I am Sam}) = P(\text{I} \mid \langle s \rangle) P(\text{am} \mid \text{I}) P(\text{Sam} \mid \text{am}) = \frac{2}{3} \frac{2}{3} \frac{1}{2} = \frac{2}{9}$$

For a *complete sentence* we would also need an end marker `</s>` and a final factor $P(\langle /s \rangle \mid \text{Sam})$. Without it, the sentence probabilities of a given length sum to one, rather than over all sentences (J&M ch. 3, footnote 1).

Smoothing for Bigrams

Add-1 / Add-k:

$$P_{\text{Add-k}}(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + k}{\text{count}(w_{i-1}) + k|V|}$$

For example:

$$P(\text{I am green}) = P(\text{I} \mid \langle s \rangle)P(\text{am} \mid \text{I})P(\text{green} \mid \text{am}) = \frac{2}{3} \frac{2}{3} \frac{0}{2} = 0$$

Using add-1, with $|V| = 10$:

$$P(\text{I am green}) = P(\text{I} \mid \langle s \rangle)P(\text{am} \mid \text{I})P(\text{green} \mid \text{am}) = \frac{2+1}{3+10} \frac{2+1}{3+10} \frac{0+1}{2+10}$$

Smoothing for Bigrams

(Stupid) Backoff: when a bigram is unseen (count = 0), rather than assigning it zero probability, we back off to the unigram score, weighted by a fixed factor λ .

$$P(\text{I am green}) = P(\text{I} \mid \langle s \rangle)P(\text{am} \mid \text{I})P(\text{green} \mid \text{am}) = \frac{2}{3} \frac{2}{3} \frac{0}{2} = 0$$

Using stupid back-off, with $\lambda = 0.4$:

$$S(\text{I am green}) = P(\text{I} \mid \langle s \rangle)P(\text{am} \mid \text{I})\lambda P(\text{green}) = \frac{2}{3} \frac{2}{3} \left(0.4 \frac{1}{14} \right)$$

N-gram Model

An n -gram model assumes a statistical dependence of a word on the $n - 1$ preceding ones:

$$P(w_i \mid w_1 \dots w_{i-1}) \approx P(w_i \mid w_{i-n+1} \dots w_{i-1})$$

The probability of the word w_i given all the previous words is approximated by the probability of w_i given only the previous $n - 1$ words.

$$\begin{aligned} P(D) &= P(w_1, w_2, \dots, w_T) \approx P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2) \dots P(w_T \mid w_{T-n+1}, \dots, w_{T-1}) \\ &\stackrel{n\text{-gram}}{=} \prod_{i=1}^T P(w_i \mid w_{i-n+1}, w_{i-n+2}, \dots, w_{i-1}) \end{aligned}$$

where T is the length of the text, and the first $n - 1$ positions are padded with $\langle s \rangle$ so that the product can start at $i = 1$.

N-Gram Model: Examples

3-gram model with 4 words ($n = 3$, $T = 4$):

$$P(w_1, w_2, w_3, w_4) \approx P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \cdot P(w_4 \mid w_2, w_3)$$

$$P(\text{I like pizza today}) \approx P(\text{I}) \cdot P(\text{like} \mid \text{I}) \cdot P(\text{pizza} \mid \text{I, like}) \cdot P(\text{today} \mid \text{like, pizza})$$

4-gram model with 6 words ($n = 4$, $T = 6$), i.e.

$$P(D) \approx \prod_{i=1}^6 P(w_i \mid w_{i-3}, w_{i-2}, w_{i-1}):$$

$$\begin{aligned} P(\text{I really like pizza with friends}) &\approx P(\text{I}) \cdot P(\text{really} \mid \text{I}) \cdot P(\text{like} \mid \text{I, really}) \\ &\cdot P(\text{pizza} \mid \text{I, really, like}) \cdot P(\text{with} \mid \text{really, like, pizza}) \cdot P(\text{friends} \mid \text{like, pizza, with}) \end{aligned}$$

How Long Should the N-Gram Be?

Word probability depends on history. Some combinatorial properties of words are fixed by **grammar**: a verb requires a subject, an article introduces a noun phrase, which is precisely the unigram independence assumption failing. Conditioning on previous words is worth paying for.

But history is expensive. The longer the n -gram:

- the more semantics and context it captures
- the **rarer** each context becomes: every estimate rests on fewer examples, giving memorization rather than generalization

So n is a compromise.

Note "*the big dog*": the article-noun dependency is already invisible to a bigram. Longer n -grams buy exactly this, and pay for it in sparsity.

Evaluation of Language Models

There are two main ways to evaluate how good a language model is:

- **Extrinsic evaluation:** test the model on an external task
- **Intrinsic evaluation:** test the model directly, without embedding it in another task

	Extrinsic Evaluation	Intrinsic Evaluation
	Test the model on another task	Test the LM directly
Question	Does it improve a real application?	How well does it predict language?
Example	Spam classification: compare stemming vs. lemmatization by their effect on classifier performance	Sentence probability: compare unigram vs. bigram by which assigns higher probability to a test sentence
Pros	measures practical usefulness	quick, cheap, task-independent
Cons	slow, task-specific	may not reflect real application performance

How Good Is Your Model? Extrinsic Evaluation

Language models usually get evaluated **extrinsically**: by the benefit they contribute to another problem, e.g. text classification, speech recognition, machine translation.

Extrinsic evaluation: embed your model in another application (e.g. a speech recognizer, a machine translation system) and see how much it improves.

But:

- **Time consuming and costly**, because it requires running multiple additional experiments.
- **Task-specific**: extrinsic evaluation may have different outcomes for different tested problems. Yet this may be desirable, because we want to find the best model *for a specific task*.

How Good Is Your Model? Intrinsic Evaluation

Intrinsic evaluation evaluates the model independently of its final application, focusing on how well the model itself captures the structure of language.

- The best model is the one that assigns the **highest probability to a test set** (based on training data).
- If additional parameters need tuning (e.g. smoothing techniques), use a development set.
- Evaluation metric: **perplexity**.

Perplexity

Once a probabilistic model is *fit* on training data, we can use test data from the same source to evaluate how good it is at predicting probabilities.

Perplexity measures how surprised a model is to observe text. Given a test text $D = w_1 w_2 \dots w_T$, its perplexity is the length-normalized inverse probability:

$$PP(D) = \sqrt[T]{\frac{1}{P(w_1 w_2 \dots w_T)}}$$

The T -th root is what makes perplexity comparable across texts of different lengths, without it, longer texts would always look "harder".

- **High perplexity:** the model is surprised by the text (low probability)
- **Low perplexity:** the model predicts the text well (high probability), which is desirable when the test data comes from the same distribution as the training

Evaluation: Perplexity

The numeric value of perplexity can be interpreted as the **average branching factor**. The branching factor of a language is the number of possible next words that can follow any word.

- e.g. a perplexity of 150 means that when the model predicts the next word, it is like choosing between 150 words with uniform probability (note that the actual vocabulary is usually much larger).

n -gram models have **lower perplexity** than unigram models,

- but when n gets too high it gets worse again, due to overfitting on the training data, i.e., the model tend to just reproduce the training set.

Limitations of N-Gram Language Models

- **Sparsity.** Most n -grams are never observed, and the problem worsens as n grows. Smoothing patches it, it does not solve it.
- **No generalization across similar words.** Having seen "*the black cat*" tells the model nothing about "*the black dog*", words are atomic symbols with no notion of similarity.
- **Fixed, short context.** Any dependency longer than $n - 1$ words is invisible, so agreement and long-range coherence are out of reach.
- **Model size grows with n .** The number of parameters is $O(|V|^n)$.

Each of these is a motivation for what comes later in the course: **embeddings** give words a notion of similarity, **recurrent models** and then **transformers** remove the fixed-context limit.

Collocations and Association Measures

From N-grams to Meaningful Multi-Word Units

n -grams capture local word sequences, but not necessarily **meaningful** combinations.

- e.g. *"the red"*, *"in the"*, *"and the"*; frequent, but not informative.

Some word sequences occur together **more often than expected by chance**, and form expressions (**collocations**) with a distinct semantic meaning.

- e.g. *"strong tea"*, *"data science"*, *"New York"*, *"break a leg"*.

Detecting these collocations is a step toward **terminology extraction** and semantic understanding.

Collocations & Terminology Extraction

Collocations are combinations of words that frequently appear together and convey a meaning that is **not purely compositional**.

- *"strong tea"* (but not *"powerful tea"*), *"break a leg"*, *"data science"*, *"Star Wars"*, *"Formula 1"*, ...

Terminology extraction focuses on identifying domain-specific multi-word units.

- e.g. *"support vector machine"*, *"named entity recognition"*.

A simple automatic recognition of collocations and multi-word terms can be achieved by detecting **statistical anomalies in word co-occurrence frequencies**, i.e., words appearing together more often than expected under an assumption of independence.

Frequency Alone Is Not Enough

The simplest method is counting: if two words occur together often, that is evidence of a special function. Ranking the bigrams of a corpus by raw frequency gives:

$\text{count}(w_1 w_2)$	w_1	w_2
80871	<i>of</i>	<i>the</i>
58841	<i>in</i>	<i>the</i>
26430	<i>to</i>	<i>the</i>
21842	<i>on</i>	<i>the</i>
18568	<i>and</i>	<i>the</i>
15494	<i>to</i>	<i>be</i>
11428	<i>New</i>	<i>York</i>

Every pair but one is made of **function words**. Frequency finds what is common, and what is common in any corpus is grammar, not terminology.

New York is the only genuine collocation in the table.

Frequency Plus a Part-of-Speech Filter

One heuristic rescues counting almost entirely: keep only candidates whose **POS pattern** looks like a phrase.

pattern	example
A N	<i>linear function</i>
N N	<i>regression coefficients</i>
A A N	<i>Gaussian random variable</i>
A N N	<i>cumulative distribution function</i>
N A N	<i>mean squared error</i>
N N N	<i>class probability function</i>
N P N	<i>degrees of freedom</i>

With the filter, the same corpus returns
New York, United States, Los Angeles, Saudi Arabia, vice president, Persian Gulf, San Francisco, Middle East, Saddam Hussein, Soviet Union.

Collocations at a Variable Distance

Frequency counting works for fixed phrases. Many collocations are looser:

she knocked on his door *they knocked at the door*

The intervening words vary and the distance is not constant, so no fixed bigram will find this. Yet there is enough regularity to tell us that *knock* is the right verb in English here, not *hit*, *beat* or *rap*.

The fix is to count inside a **context window** rather than at a fixed offset, and to look at the **mean and variance** of the distance between the two words:

- a small variance means a rigid relationship, so the pair is worth reporting
- a large variance means the two words merely co-occur in the same region, with no fixed structure
- **larger** windows → *topical* association; **smaller** windows → *syntactic* relations

The Null Hypothesis for Collocations

We formulate a null hypothesis H_0 : the two words are **independent**, and any co-occurrence we see is chance.

$$H_0 : P(w_1 w_2) = P(w_1) P(w_2)$$

An association measure asks how far the **observed** co-occurrence departs from what H_0 predicts. If the departure is large enough, we reject H_0 and call the pair a collocation.

Every measure in this section is a different answer to "*how far is far enough?*", and they disagree in a specific way: some reward **rare and surprising** pairs, others reward **frequent and reliable** ones.

This is the same independence assumption that the unigram model made about whole sentences at the start of this lecture. Here we test it on a single pair, instead of assuming it.

Pointwise Mutual Information

H_0 says the pair should occur with probability $P(w_1)P(w_2)$. So compare that with how often it **actually** occurs, as a ratio, and take a logarithm so independence sits at zero:

$$\text{PMI}(w_1, w_2) = \log_2 \frac{P(w_1 w_2)}{P(w_1) P(w_2)} = \log_2 \frac{|C| \text{count}(w_1 w_2)}{\text{count}(w_1) \text{count}(w_2)}$$

Both forms are the same number: the second is just the first with every probability replaced by its **maximum likelihood estimate***.

- $\text{PMI} > 0$: the pair occurs more often than chance predicts
- $\text{PMI} = 0$: the two words are independent
- $\text{PMI} < 0$: the pair occurs less often than chance predicts

* $P(w_1 w_2) = \text{count}(w_1 w_2)/|C|$ and $P(w_i) = \text{count}(w_i)/|C|$

PMI Favors Rare Words

Take the most strongly associated pair imaginable: two words that **only ever occur together**. Every w_1 is followed by w_2 , and every w_2 is preceded by w_1 .

Then every occurrence of either word is part of the pair, so all three counts are the same number. Call it c :

$$\text{count}(w_1) = \text{count}(w_2) = \text{count}(w_1w_2) = c$$

Substituting into PMI:

$$\text{PMI}(w_1, w_2) = \log_2 \frac{|C| c}{c \cdot c} = \log_2 \frac{|C|}{c}$$

The association has **cancelled out completely**. What is left *inversely* depends only on c .

PMI Favors Rare Words

So among pairs that always occur together, the **rarer one scores higher**:

pair, in a corpus of $ C = 10^6$ tokens	c	PMI
seen twice , always together	2	18.9
seen two thousand times , always together	2000	9.0

- Both pairs are perfectly associated, so the quantity PMI is meant to measure is *identical* for the two. Everything separating them is rarity, and PMI ranks the pair with **two** observations twice as high as the one with two thousand.
- PMI rewards **surprise**, not **reliability**. Two observations are not evidence of anything, but PMI cannot tell, because the corpus size is the only thing it has left to look at.

Other Association Measures

PMI is not the only way to compare what we observe with what H_0 predicts. The classical alternatives come from **statistical hypothesis testing**:

measure	what it asks	what it rewards
t -score	is the excess large <i>relative to how often we saw the pair</i> ?	pairs that are frequent and consistent , never a pair seen twice
χ^2	is this whole pattern of counts unlikely under independence?	any clear departure from independence , in either direction
likelihood ratio	how many times more likely is dependence than independence?	the same, but judged more reliably on rare pairs , which is most pairs

Each carries a significance threshold from its own distribution, so "*is this a collocation?*" becomes a decision and not only a ranking.

From PMI to Positive Pointwise Mutual Information (PPMI)

PMI can take **negative values**, i.e. when two words co-occur *less* often than expected under independence.

- However, negative PMI values are often not meaningful in applications such as collocation extraction or vector representations:
 - They indicate *lack of association*, not useful evidence of dissimilarity.
 - They can also bias the distribution of feature values in downstream models.

To address this, we use the **Positive Pointwise Mutual Information (PPMI)**:

$$\text{PPMI}(w_1, w_2) = \max(\text{PMI}(w_1, w_2), 0)$$

- Keeps only positive associations between words, setting all negative PMI values to zero.
 - (*coffee*, *cup*): $\text{PMI} = +2.3 \rightarrow \text{PPMI} = 2.3$
 - (*coffee*, *keyboard*): $\text{PMI} = -1.8 \rightarrow \text{PPMI} = 0$

Collocation Extraction in Summary

Every implementation follows the same five steps. What changes between them is the scoring function and the defaults.

1. **Count.** Build two frequency distributions over the corpus: one for single tokens, one for the pairs that co-occur inside the context window.
2. **Prune by frequency.** Discard pairs seen fewer than *min count* times. Rare pairs give unstable scores, and this is also what keeps the counting tractable.
3. **Filter.** Optionally drop pairs containing stopwords or function words, or restrict to chosen POS patterns such as *adjective + noun*.
4. **Score and rank.** Apply an association measure to every surviving pair, then sort.
5. **Threshold.** Keep the pairs above a cutoff and merge each one into a **single token**.

The two decisions that matter are the **context window** and the **association measure**. Everything else is bookkeeping.

Probability

- Yan, L. & Cain, J., *Conditional Probability*. Stanford CS109, pg. 33-61.

Language models

- Jurafsky & Martin, *Speech and Language Processing*, 3rd ed., ch. 3.

Collocations and association measures

- Manning & Schütze, *Foundations of Statistical NLP*, ch. 5. MIT Press, 1999.
- Jurafsky & Martin, *Speech and Language Processing*, 3rd ed., Appendix J.
- Church & Hanks, *Word association norms, mutual information, and lexicography*. Computational Linguistics 16(1), 1990.